

PROJECT REPORT

Automated Deployment System for React Application

CI/CD Pipeline with Docker, Jenkins, AWS EC2, and Prometheus–Grafana Monitoring

Submitted by

M.AKSHAYA

Prepared as part of DevOps Training
GitHub Repository: Automated-React-Deployment
September 2026

1. Overview

This project implements an end-to-end automated deployment workflow for a React web application. The system builds, containerizes, and deploys the application to an AWS EC2 instance using Docker, Docker Hub, and Jenkins, with full infrastructure monitoring and alerting through Prometheus, Grafana, and Node Exporter.

When the Jenkins pipeline is triggered for the dev branch, the following sequence takes place:

1. Checks out the source code from GitHub.
2. Builds a Docker image of the React application.
3. Pushes the image to Docker Hub.
4. Deploys the image to an AWS EC2 instance.
5. Runs the React application inside an Nginx container.

The deployed application is continuously monitored using Prometheus and Grafana, with automated email alerts configured for high CPU usage.

2. System Architecture

The overall deployment architecture connects the developer, source control, CI/CD pipeline, container registry, and cloud infrastructure into a single automated flow:



Flow Summary:

GitHub → Jenkins → Docker Build → Docker Hub → AWS EC2 → React Application

3. Technology Stack

The project uses the following tools and technologies across development, deployment, and monitoring:

Category	Technology
Frontend	React
Web Server	Nginx
Containerization	Docker
Image Registry	Docker Hub
CI/CD	Jenkins
Cloud Hosting	AWS EC2
Operating System	Amazon Linux 2023
Version Control	Git and GitHub
Remote Access	SSH
Metrics Collection	Node Exporter
Monitoring	Prometheus
Visualization / Dashboards	Grafana
Alerting	Grafana Alerting + Gmail (SMTP)

4. Project Structure

The repository is organized as follows:

```
Automated-React-Deployment/  
├── build/  
│   ├── index.html  
│   ├── static/  
│   └── assets/  
├── Dockerfile  
├── Jenkinsfile  
├── README.md  
├── react_app_cicd_architecture.png  
├── docs/  
│   └── screenshots/
```

5. Containerization with Docker

The React application is containerized using a lightweight Nginx-based Dockerfile:

```
FROM nginx:alpine  
  
COPY build /usr/share/nginx/html  
  
EXPOSE 80
```

Dockerfile Explanation

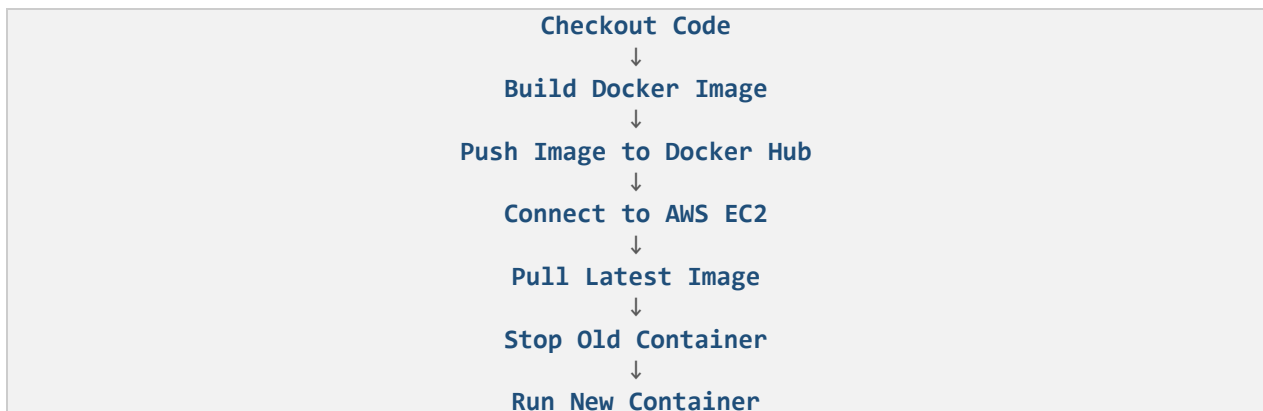
- FROM nginx:alpine — uses the lightweight Nginx Alpine base image.
- COPY build /usr/share/nginx/html — copies the React production build files into the Nginx web directory.
- EXPOSE 80 — documents that the application serves traffic on port 80 inside the container.

6. Jenkins CI/CD Pipeline

The Jenkins pipeline automates the build and deployment process through the following stages:

6. Checkout Code — retrieves the latest code from the GitHub dev branch.
7. Build Docker Image — builds the Docker image for the React application.
8. Push to Docker Hub — pushes the built image to Docker Hub using stored Jenkins credentials.
9. Deploy to EC2 — connects to the EC2 instance, pulls the latest image, and runs the container.

Pipeline Flow



7. Local Setup and Usage

Prerequisites

- Git
- Docker Desktop
- A Docker Hub account
- A working React build folder

Clone the Repository

```
git clone https://github.com/Akshaya33-Devops/Automated-React-Deployment.git
cd Automated-React-Deployment
```

Build and Run the Docker Image

```
docker build -t react-app .
docker run -d -p 8080:80 --name react-container react-app
```

The application can then be accessed locally at:

```
http://localhost:8080
```

8. AWS EC2 Deployment

8.1 Launch an EC2 Instance

- Operating System: Amazon Linux 2023
- Instance Type: t3.micro
- SSH Port: 22
- HTTP Port: 80

8.2 Install Docker on EC2

```
sudo dnf update -y
sudo dnf install docker -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
```

8.3 Pull, Stop, and Run the Container

```
docker pull akshayanimuthu/react-app:dev
docker stop react-container || true
docker rm react-container || true
docker run -d \
  -p 80:80 \
  --name react-container \
  --restart unless-stopped \
  akshayanimuthu/react-app:dev
```

8.4 Access the Deployed Application

```
http://13.201.83.88
```

9. Monitoring

The deployed React application and its host EC2 instance are monitored using Node Exporter, Prometheus, and Grafana.

Monitoring Flow



↓
Grafana

Node Exporter

Node Exporter collects system-level metrics from the AWS EC2 instance, including CPU, memory, disk, and network usage. The Node Exporter target is configured at localhost:9100.

Prometheus

Prometheus scrapes and stores the metrics exposed by Node Exporter. The Prometheus target health was verified as UP.

Grafana

Grafana is connected to Prometheus as its data source. The “EC2 Monitoring” dashboard includes panels for:

- Target Status
- CPU Usage
- Memory Usage
- Disk Usage
- Network Received
- Network Transmitted

10. Alerting and Email Notification

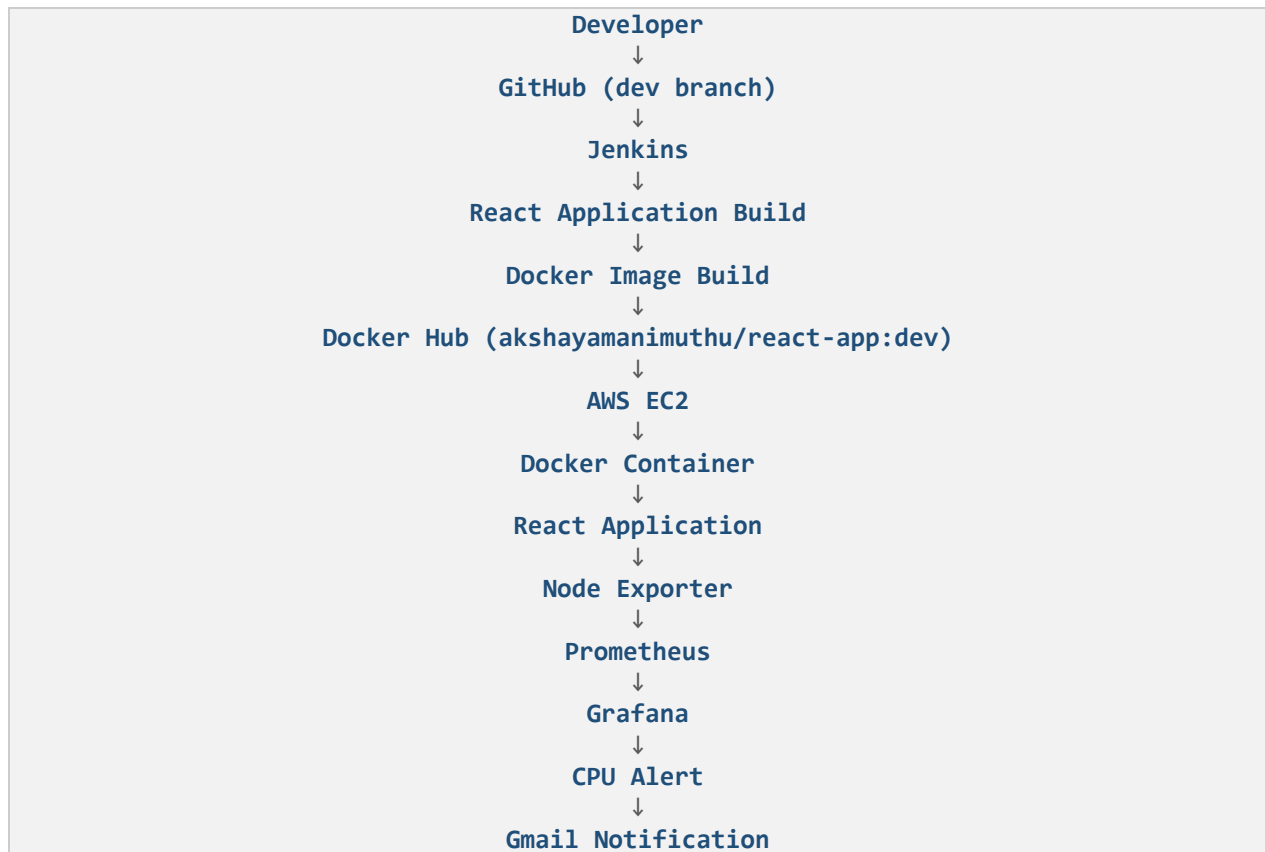
A Grafana alert rule was configured to monitor high CPU usage on the EC2 instance:

Setting	Value
Alert Name	High CPU Usage
CPU Threshold	80%
Pending Period	5 minutes
Notification Channel	Email (Gmail SMTP)

The alert was tested successfully, and a test notification was received via Gmail. This enables the system administrator to be notified automatically whenever CPU usage remains above the configured threshold for the specified duration.

11. Full Project Architecture (CI/CD + Monitoring)

The complete workflow, from source code commit to live monitoring and alerting, is summarized below:



12. Monitoring Health Check

The monitoring setup was verified using the following checks:

- ✓ Node Exporter service: Running
- ✓ Prometheus service: Running
- ✓ Prometheus target: UP
- ✓ Grafana service: Running
- ✓ Grafana dashboard: Available
- ✓ High CPU alert: Tested successfully
- ✓ Gmail notification: Received successfully

13. CI/CD Setup and Security Notes

Docker Hub

The Docker image is pushed to the following Docker Hub repository, tagged as dev:

```
akshayanimuthu/react-app:dev
```

Docker Hub credentials are stored securely in Jenkins and are never committed to GitHub.

Jenkins Credentials

The pipeline uses the following credentials, configured securely within Jenkins:

- dockerhub-credentials
- ec2-ssh-key

EC2 SSH Access

Jenkins requires SSH access to the EC2 instance, and the private key is stored securely in Jenkins credentials. The following are never committed to GitHub:

- Private SSH keys
- AWS access keys
- Docker Hub passwords and access tokens
- .pem files
- Secret configuration files

14. Project Screenshots

Supporting screenshots, stored in docs/screenshots/, document each stage of the pipeline and monitoring setup:

File	Description
aws-ec2-console.png	AWS EC2 console showing the running instance
aws-security-group-configuration.png	EC2 security group inbound rules (SSH, HTTP, etc.)
deployed-react-application.png	React application running via the EC2 public IP
dockerhub-repository-image-tags.png	Docker Hub repository showing the dev image tag
grafana-dashboard.png (views 1 & 2)	Grafana EC2 Monitoring dashboard
grafana-email-notification.png	Email notification received from Grafana alert
grafana-high-cpu-alert.png	Grafana High CPU Usage alert configuration
jenkins-configuration-settings.png	Jenkins pipeline/job configuration settings
jenkins-login-page.png	Jenkins login page
jenkins-pipeline-execution.png	Successful Jenkins pipeline run
prometheus-target-up.png	Prometheus target showing Node Exporter as UP

15. Deployment Verification Checklist

- ✓ Docker image built successfully.
- ✓ React application tested locally.
- ✓ Docker image pushed to Docker Hub.
- ✓ Docker Hub image tagged as dev.
- ✓ Docker installed on EC2.
- ✓ React container running on EC2.
- ✓ Application accessed through the EC2 public IP.
- ✓ Jenkins pipeline completed successfully.
- ✓ Node Exporter installed and running on EC2.
- ✓ Prometheus configured and target verified as UP.
- ✓ Grafana dashboard created and connected to Prometheus.
- ✓ High CPU alert configured and tested.
- ✓ Email notification received via Gmail.

16. Future Improvements

- Add automated testing.
- Add SonarQube code-quality analysis.
- Add HTTPS using a domain name and SSL certificate.
- Add health checks.
- Add rollback support.
- Add separate staging and production branches.
- Use environment variables for deployment configuration.
- Expand monitoring with additional Grafana dashboards (disk I/O, container-level metrics).
- Add alerting for memory and disk usage thresholds.
- Integrate Alertmanager for more advanced alert routing (Slack, PagerDuty, etc.).

17. Conclusion

This project successfully demonstrates a complete DevOps workflow — from source code management through automated build, containerization, deployment, and infrastructure monitoring. By integrating GitHub, Jenkins, Docker, Docker Hub, AWS EC2, and the Prometheus–Grafana monitoring stack, the system achieves a reliable, repeatable, and observable deployment pipeline for a React application, with proactive alerting to support timely operational response.

18. Project Links

- GitHub Repository: github.com/Akshaya33-Devops/Automated-React-Deployment (dev branch)
- Docker Hub: hub.docker.com/u/akshayanimuthu
- Docker Image: [akshayanimuthu/react-app:dev](#)
- Live Application: <http://13.201.83.88>

19. License

This project is created for educational and portfolio purposes as part of DevOps training.

— End of Report —